

Workspeed

Documentação Técnica — Estrutura do MVP

Este documento detalha, módulo a módulo, o escopo funcional do MVP do sistema ERP e define a ficha de informações necessárias (campos de dados) para cada entidade envolvida. O objetivo é servir como base para modelagem de banco de dados, definição de formulários e planejamento do desenvolvimento.

Convenções utilizadas nas fichas: o campo "Tipo" indica o tipo de dado esperado (texto, número, data, decimal/moeda, booleano, seleção ou relacionamento). O campo "Obrigatório" indica se o preenchimento é necessário no cadastro. A coluna "Observações" traz regras de negócio, valores padrão ou dependências.

Arquitetura de Multi-tenancy

O sistema é multiempresa (multi-tenant): a mesma aplicação atende várias empresas (tenants), cada uma enxergando apenas os seus próprios dados. Existem duas estratégias principais para implementar isso em um banco PostgreSQL, descritas a seguir.

Modelo 1 — tenant_id (coluna compartilhada)

Todas as empresas dividem as mesmas tabelas, no mesmo schema. Uma coluna tenant_id em cada tabela indica a qual empresa aquele registro pertence. Toda consulta ao banco precisa filtrar por esse campo.

Exemplo — tabela produtos com duas empresas convivendo na mesma tabela:

id	tenant_id	nome	preco
1	1	Cimento	35.00
2	1	Tijolo	0.80
3	2	Notebook	3500.00
4	2	Mouse	45.00

Na prática, o NestJS identifica a empresa logada (via JWT) em um middleware/guard e aplica o filtro tenant_id automaticamente em todas as queries, para evitar que uma empresa acesse dado de outra.

Modelo 2 — Schema por tenant

Cada empresa recebe seu próprio schema dentro do mesmo banco de dados, com as mesmas tabelas replicadas por schema. O isolamento é físico, não apenas lógico.

Exemplo de organização:

- banco_erp → schema empresa_1 (produtos, pedidos, clientes...)
- banco_erp → schema empresa_2 (produtos, pedidos, clientes...)
- banco_erp → schema public (tabelas globais: empresas, planos)

Exige um middleware que resolva o tenant por requisição (subdomínio, header ou JWT) e troque o search_path da conexão antes de cada query, além de rodar as migrations em cada schema sempre que uma empresa nova é criada.

Comparativo entre os modelos

Critério	Modelo tenant_id	Modelo schema por tenant
Complexidade de implementação	Baixa	Alta
Isolamento de dados	Lógico (mesma tabela)	Físico (schemas separados)
Risco de vazamento entre empresas	Maior, se o filtro for esquecido	Menor: schema errado nem existe para o tenant
Migrations	Executadas uma única vez	Precisam rodar em cada schema
Exclusão/backup de um cliente	Mais trabalhoso (precisa filtrar)	Simples: basta remover o schema
Curva de aprendizado	Menor	Maior: AsyncLocalStorage/RequestContext + DataSource dinâmico

Recomendação para este projeto

Como o desenvolvimento está sendo feito de forma gradual e o schema-por-tenant tem uma complexidade real no NestJS/TypeORM (troca dinâmica de search_path, migrations por schema, gerenciamento de

conexões), uma alternativa é começar o MVP com `tenant_id` e migrar para `schema-por-tenant` depois, quando o volume de empresas ou uma exigência de isolamento físico justificar a mudança. Caso a decisão seja seguir direto com `schema-por-tenant`, o ponto de maior atenção é a resolução do tenant por requisição, tratada em detalhe na próxima seção.

Resolução do tenant por requisição (schema-per-tenant)

Passos necessários para implementar esse modelo no NestJS:

- 1. Middleware identifica o tenant (subdomínio, header customizado ou claim no JWT) antes da rota ser processada.
- 2. O tenant identificado é armazenado no contexto da requisição (ex.: `AsyncLocalStorage` ou biblioteca como `nestjs-cls`), acessível em qualquer service durante aquela requisição.
- 3. Um `DataSource/QueryRunner` do `TypeORM` aplica o `search_path` correspondente ao schema do tenant antes de executar as queries.
- 4. Toda vez que uma empresa nova é cadastrada, um processo automatizado cria o schema e roda as migrations nele — isso não pode ser manual, senão vira gargalo operacional conforme a base de clientes cresce.

Arquitetura de Funcionalidades por Empresa (Feature Flags)

Definido o modelo de multi-tenancy por tenant_id, este projeto adota também um segundo eixo de arquitetura: cada empresa (tenant) pode ter um conjunto diferente de funcionalidades habilitadas, mantendo um único backend e um único frontend para todos os clientes.

Exemplo prático:

- Empresa A: Financeiro, Produtos, Clientes, Integração Nubank
- Empresa B: Financeiro, Produtos, Clientes, Integração Serasa

Ambas as empresas usam o mesmo sistema, mas cada uma enxerga apenas os módulos contratados. O objetivo é evitar múltiplos backends ou múltiplas versões do ERP.

Conceito — Feature Flags por Tenant

Esse padrão, conhecido como Feature Flags por Tenant, se apoia em quatro princípios:

- Existe apenas um backend.
- Existe apenas um frontend.
- Todas as funcionalidades são desenvolvidas uma única vez.
- O que muda entre empresas é apenas quais funcionalidades cada uma possui habilitadas.

Modelagem do banco de dados

Ficha de informações — Empresa

Campo	Tipo	Obrigatório	Observações
id	Número (PK)	Sim	Identificador único da empresa
razaoSocial	Texto	Sim	Nome/razão social da empresa (tenant)

Ficha de informações — Feature

Campo	Tipo	Obrigatório	Observações
id	Número (PK)	Sim	Identificador único da funcionalidade
codigo	Texto	Sim	Identificador único usado no código. Ex.: financeiro, produto, cliente, integracao_nubank, integracao_serasa, dashboard, insights
nome	Texto	Sim	Nome de exibição da funcionalidade

Exemplo de entidades em NestJS/TypeORM:

```
@Entity('empresa')
class Empresa {
  id
  razaoSocial
}

@Entity('feature')
class Feature {
  id
  codigo
  nome
}
```

Tabela intermediária — empresa_features

A relação entre empresas e funcionalidades é Many-to-Many, resolvida por uma tabela intermediária:

empresa_id	feature_id
1	financeiro
1	produto
1	cliente
1	integracao_nubank
2	financeiro
2	produto
2	cliente
2	integracao_serasa

Esse modelo é mais escalável do que criar colunas booleanas na tabela empresa (ex.: temFinanceiro, temNubank, temSerasa). Colunas booleanas exigem alterar a estrutura da tabela e rodar uma migration a cada nova funcionalidade criada; a tabela empresa_features permite adicionar, remover ou combinar funcionalidades por empresa sem nenhuma alteração de schema — apenas inserindo ou removendo registros.

Backend (NestJS) — fluxo de identificação de features

- 1. Usuário realiza login.
- 2. Backend valida o JWT.
- 3. Identifica o usuário.
- 4. Identifica a empresa vinculada ao usuário.
- 5. Busca todas as Features daquela empresa (via empresa_features).
- 6. Retorna ao frontend a lista de features habilitadas.

Exemplo de retorno do endpoint `GET /auth/me` (ou `GET /users/me`):

```
{
  "id": 15,
  "nome": "Eduardo",
  "empresa": {
    "id": 2,
    "nome": "AgroMais"
  },
  "features": [
    "financeiro",
    "produto",
    "cliente",
    "integracao_nubank"
  ]
}
```

Proteção das rotas

Esconder um menu ou botão no frontend não é segurança — qualquer pessoa pode chamar a API diretamente. Toda rota do backend precisa validar, de forma independente, se a empresa do usuário possui a feature correspondente.

Fluxo de validação:

- 1. Requisição chega em `GET /financeiro`.
- 2. Backend valida se o JWT é válido.
- 3. Backend verifica se a empresa possui a feature financeiro.
- 4. Se sim, libera o acesso; se não, retorna HTTP 403 Forbidden.

Exemplo simples de verificação:

```
empresa.features.some(
  feature => feature.codigo === "financeiro"
)
```

Esse trecho pode evoluir futuramente para um FeatureGuard personalizado no NestJS, usando decorators para proteger rotas de forma declarativa:

```
@Feature("financeiro")
@Get()
findAll() {}
```

Guards no NestJS — AuthGuard e FeatureGuard

No NestJS, a proteção de rota é feita com Guards, aplicados através do decorator `@UseGuards()`. Existem dois tipos de verificação em jogo, e cada um deve ficar em um guard separado:

- Autenticação — responde "esse JWT é válido? quem é esse usuário?". É responsabilidade do AuthGuard.
- Autorização — responde "esse usuário, mesmo autenticado, pode acessar isso?". É responsabilidade do FeatureGuard.

O AuthGuard pode ser aplicado uma única vez no topo do controller, valendo para todas as rotas dele, em vez de repetir em cada método:

```
@UseGuards(AuthGuard)
@Controller('financeiro')
export class FinanceiroController {

  @Get()
  findAll() { ... }

  @Post()
  create() { ... }
}
```

Combinando AuthGuard e FeatureGuard no mesmo controller:

```
@UseGuards(AuthGuard, FeatureGuard)
@Feature('financeiro')
@Controller('financeiro')
export class FinanceiroController {
  ...
}
```

A ordem dos guards no array importa: o AuthGuard precisa vir antes do FeatureGuard, porque o FeatureGuard depende do `request.user`, que só é preenchido depois que o AuthGuard valida o JWT.

Esboço do FeatureGuard: ele lê o código da feature exigida (a partir do decorator `@Feature`, via `Reflector`) e compara com as features da empresa do usuário autenticado:

```
@Injectable()
export class FeatureGuard implements CanActivate {
  constructor(private reflector: Reflector) {}

  canActivate(context: ExecutionContext): boolean {
    const featureCodigo = this.reflector.get<string>(
      'feature',
      context.getHandler(),
    );
    const { user } = context.switchToHttp().getRequest();

    return user.empresa.features.some(
      f => f.codigo === featureCodigo,
    );
  }
}
```

```
    );  
  }  
}
```

Quando `canActivate` retorna `false`, o NestJS responde automaticamente com HTTP 403 Forbidden, sem executar o método da rota.

Frontend

No frontend, o objetivo não é reimplementar a segurança (isso já é papel do backend), mas sim usar as features para melhorar a experiência: montar o menu certo, evitar chamadas desnecessárias e redirecionar o usuário quando ele tenta acessar algo que não tem.

Após o login, o frontend segue estes passos:

- 1. Salva o JWT recebido (ex.: em memória ou em um cookie `httpOnly`, evitando `localStorage` quando possível por questões de segurança contra XSS).
- 2. Chama `GET /auth/me`.
- 3. Recebe a lista de features habilitadas para a empresa do usuário.
- 4. Armazena essas features no estado global da aplicação.
- 5. Enquanto essa chamada não retorna, exibe um estado de carregamento em vez de renderizar o menu vazio ou incompleto.

Exemplo de resposta armazenada no estado global:

```
{  
  "features": [  
    "financeiro",  
    "produto",  
    "cliente"  
  ]  
}
```

O gerenciamento desse estado global pode ser feito com Context API, Redux, Zustand, Pinia ou outra biblioteca equivalente, conforme o stack do frontend escolhido. Um padrão comum em React é expor um hook customizado que encapsula a verificação, para não espalhar `features.includes(...)` por toda a aplicação:

```
function useFeature(codigo: string): boolean {  
  const { features } = useAuth();  
  return features.includes(codigo);  
}  
  
function PaginaFinanceiro() {  
  const temFinanceiro = useFeature('financeiro');  
  
  if (!temFinanceiro) {  
    return <Navigate to="/nao-autorizado" />;  
  }  
  
  return <Financeiro />;  
}
```

Esse componente de proteção (equivalente a um Guard, só que no frontend) evita que o usuário fique numa tela quebrada quando digita a URL de um módulo que a empresa não contratou. Ele redireciona para uma página amigável — mas o bloqueio real de dados continua sendo feito pelo backend, através do FeatureGuard.

Ponto de atenção: como o JWT tem prazo de expiração e as features podem mudar (ex.: empresa contratou um novo módulo), vale revalidar o `GET /auth/me` periodicamente ou a cada novo login, em vez de confiar apenas no valor salvo no primeiro carregamento da aplicação.

Menu dinâmico

O menu lateral não é fixo: ele é montado dinamicamente a partir das features retornadas para a empresa logada.

Exemplo — Empresa A:

- Dashboard
- Financeiro
- Produtos
- Clientes
- Nubank

Exemplo — Empresa B:

- Dashboard
- Financeiro
- Produtos
- Clientes
- Serasa

Pseudocódigo de montagem do menu:

```
if (features.includes("financeiro")) {  
  mostrarMenuFinanceiro()  
}  
  
if (features.includes("integracao_serasa")) {  
  mostrarMenuSerasa()  
}
```

Fluxo completo

- 1. Login
- 2. Backend gera o JWT
- 3. Frontend chama GET /auth/me
- 4. Backend valida o token
- 5. Backend busca a empresa vinculada ao usuário
- 6. Backend busca os registros de empresa_features
- 7. Backend retorna a lista de funcionalidades habilitadas
- 8. Frontend monta os menus dinamicamente
- 9. Usuário acessa um módulo
- 10. Backend valida novamente a feature naquela rota
- 11. Backend permite ou bloqueia o acesso (200 ou 403)

Theming por tenant (personalização visual por empresa)

Além de controlar quais módulos cada empresa enxerga, o mesmo frontend único pode se adaptar visualmente conforme a empresa logada — prática conhecida como white-label ou theming por tenant, comum em SaaS multiempresa.

O backend retorna, junto com as features, um objeto de tema associado à empresa:

```
{  
  "empresa": {  
    "id": 2,  
    "nome": "AgroMais",
```

```
"tema": {
  "corPrimaria": "#2E7D32",
  "corSecundaria": "#1B5E20",
  "logo": "https://cdn.workspeed.com/logos/agromais.png",
  "nomeExibicao": "AgroMais Gestão"
},
"features": ["financeiro", "produto", "cliente"]
}
```

No frontend, esse tema é aplicado em tempo de execução via CSS variables, assim que o GET /auth/me retorna:

```
document.documentElement.style.setProperty('--cor-primaria',
empresa.tema.corPrimaria);
document.documentElement.style.setProperty('--cor-secundaria',
empresa.tema.corSecundaria);
```

Como todo o CSS do sistema referencia var(--cor-primaria) em vez de cores fixas, a interface inteira muda de cor sem exigir um build separado por empresa — continua sendo um único frontend, um único deploy.

Níveis de customização, em ordem de complexidade:

- Cores e logo — nível mais simples, aplicado via CSS variables; praticamente todo SaaS B2B oferece isso.
- Textos customizados — nome do sistema e nomes de módulos (ex.: uma empresa pode preferir chamar "Financeiro" de "Tesouraria").
- Layout ou estrutura diferente — nível mais complexo (ex.: sidebar vs. topbar, densidade de informação); exige componentes condicionais no código, não apenas CSS.

Recomendação: manter a personalização nos níveis de cores/logo/textos para todos os clientes, por serem baratos de manter. Mudanças de layout/estrutura tendem a virar dívida técnica se oferecidas a qualquer cliente que peça — geralmente fazem mais sentido reservadas a planos enterprise.

Benefícios da arquitetura

- Um único backend para todos os clientes.
- Um único frontend para todos os clientes.
- Módulos reutilizáveis entre empresas.
- Funcionalidades exclusivas por empresa, sem duplicar sistema.
- Facilidade para vender módulos adicionais (upsell) sem retrabalho técnico.
- Código mais limpo e centralizado.
- Evita a criação de vários sistemas diferentes para clientes diferentes.
- Altamente escalável conforme a base de clientes cresce.
- Arquitetura amplamente utilizada em SaaS modernos.

1. Dashboard

Visão geral e rápida da situação da empresa, consolidando dados dos demais módulos em um único painel.

Funcionalidades

- Faturamento do mês
- Saldo em caixa
- Contas a pagar (resumo)
- Contas a receber (resumo)
- Pedidos do dia
- Estoque baixo (alerta)
- Últimas movimentações financeiras
- Alertas importantes (vindos do módulo Insights)

Ficha de informações — Configuração do Dashboard (não é uma entidade persistente própria; consome dados de outros módulos)

O Dashboard não possui cadastro próprio. Ele agrega dados de Financeiro, Pedidos, Produtos, Contas a Pagar/Receber e Insights, filtrados pelos parâmetros abaixo.

Campo	Tipo	Obrigatório	Observações
Empresa (tenant)	Relacionamento	Sim	Filtra os dados pela empresa logada, em cenário multiempresa
Período de referência	Data (intervalo)	Sim	Padrão: mês corrente; permite seleção de outro período
Usuário logado	Relacionamento	Sim	Define permissões de visualização por papel (role)
Limite de itens recentes	Número	Não	Quantidade de movimentações recentes exibidas (padrão: 10)

Indicadores do módulo

- Faturamento do mês vs. mês anterior
- Saldo em caixa consolidado
- Quantidade de pedidos do dia e valor total
- Quantidade de produtos com estoque abaixo do mínimo

2. Financeiro

Gestão de contas bancárias e movimentações financeiras: entradas, saídas e transferências entre contas.

Funcionalidades

- Cadastro de contas bancárias
- Registro de movimentações (entradas, saídas e transferências)
- Consulta de saldo por conta e saldo consolidado
- Fluxo de caixa e evolução do caixa

Ficha de informações — Conta Bancária

Campo	Tipo	Obrigatório	Observações
Banco	Relacionamento (Banco)	Sim	Vinculado ao cadastro de Bancos (módulo Cadastros)
Agência	Texto	Não	Aplicável a contas correntes
Número da conta	Texto	Sim	Identificador da conta no banco
Tipo de conta	Seleção	Sim	Corrente, Poupança ou Caixa interno
Saldo inicial	Decimal (moeda)	Sim	Saldo no momento do cadastro
Saldo atual	Decimal (moeda)	Sim (calculado)	Atualizado automaticamente pelas movimentações
Empresa (tenant)	Relacionamento	Sim	Cenário multiempresa
Status	Booleano	Sim	Ativa ou inativa

Ficha de informações — Movimentação Financeira

Campo	Tipo	Obrigatório	Observações
Data	Data	Sim	Data efetiva da movimentação
Tipo	Seleção	Sim	Entrada, Saída ou Transferência
Valor	Decimal (moeda)	Sim	Sempre positivo; o tipo define o sinal no saldo
Conta de origem	Relacionamento (Conta Bancária)	Sim	Obrigatório para saída e transferência
Conta de destino	Relacionamento (Conta Bancária)	Condicional	Obrigatório apenas para transferências
Categoria	Relacionamento (Categoria de Despesa/Receita)	Não	Usada para relatórios e indicadores
Forma de pagamento	Relacionamento	Não	Vinculada ao cadastro de Formas de Pagamento
Descrição	Texto	Não	Detalhamento livre da movimentação
Comprovante/anexo	Arquivo	Não	Upload de comprovante
Origem do registro	Seleção	Sim (calculado)	Manual, Conta a Pagar ou Conta a Receber (baixa automática)

Indicadores do módulo

- Saldo atual consolidado
- Entradas x saídas no período
- Evolução do caixa (gráfico de linha por período)

3. Contas a Pagar

Controle de obrigações financeiras da empresa junto a fornecedores e terceiros.

Funcionalidades

- Cadastro de contas e parcelamento
- Baixa de pagamento (total ou parcial)
- Controle de vencimentos
- Histórico de pagamentos

Ficha de informações — Conta a Pagar

Campo	Tipo	Obrigatório	Observações
Fornecedor	Relacionamento (Fornecedor)	Sim	Pode ser opcional para despesas avulsas
Descrição	Texto	Sim	Ex.: Aluguel, Compra de mercadorias
Categoria de despesa	Relacionamento	Não	Usada em indicadores de despesas por categoria
Valor total	Decimal (moeda)	Sim	Valor total da obrigação
Número de parcelas	Número	Sim	Padrão: 1 (à vista)
Data de emissão	Data	Sim	Data de lançamento da conta
Documento/anexo	Arquivo	Não	Nota fiscal, boleto ou contrato
Status	Seleção	Sim (calculado)	Pendente, Parcialmente paga, Paga ou Vencida

Ficha de informações — Parcela (Conta a Pagar)

Campo	Tipo	Obrigatório	Observações
Conta a pagar (vínculo)	Relacionamento	Sim	Conta de origem
Número da parcela	Número	Sim	Ex.: 1/3, 2/3
Valor da parcela	Decimal (moeda)	Sim	Soma das parcelas deve fechar com o valor total
Data de vencimento	Data	Sim	Usada para alertas de vencimento
Data de pagamento	Data	Não	Preenchida na baixa
Forma de pagamento	Relacionamento	Não	Preenchida na baixa
Conta bancária de saída	Relacionamento	Não	Gera movimentação financeira automática na baixa
Status	Seleção	Sim (calculado)	Pendente, Paga ou Vencida

Indicadores do módulo

- Total a pagar no período
- Contas vencidas

- Contas a vencer (próximos 7/30 dias)
- Total já pago no período

4. Contas a Receber

Controle de valores a receber de clientes, incluindo os gerados automaticamente por pedidos.

Funcionalidades

- Cadastro de recebimentos e parcelamento
- Baixa de recebimento (total ou parcial)
- Controle de vencimentos
- Histórico de recebimentos

Ficha de informações — Conta a Receber

Campo	Tipo	Obrigatório	Observações
Cliente	Relacionamento (Cliente)	Sim	Vinculado ao cadastro de Clientes
Pedido de origem	Relacionamento (Pedido)	Não	Preenchido quando gerado automaticamente por um pedido
Descrição	Texto	Sim	Ex.: Venda de produtos, Serviço prestado
Valor total	Decimal (moeda)	Sim	Valor total a receber
Número de parcelas	Número	Sim	Padrão: 1 (à vista)
Data de emissão	Data	Sim	Data de lançamento
Status	Seleção	Sim (calculado)	Em aberto, Parcialmente recebida, Recebida ou Vencida

Ficha de informações — Parcela (Conta a Receber)

Campo	Tipo	Obrigatório	Observações
Conta a receber (vínculo)	Relacionamento	Sim	Conta de origem
Número da parcela	Número	Sim	Ex.: 1/2, 2/2
Valor da parcela	Decimal (moeda)	Sim	Soma das parcelas deve fechar com o valor total
Data de vencimento	Data	Sim	Usada para alertas de vencimento
Data de recebimento	Data	Não	Preenchida na baixa
Forma de recebimento	Relacionamento	Não	Preenchida na baixa
Conta bancária de entrada	Relacionamento	Não	Gera movimentação financeira automática na baixa
Status	Seleção	Sim (calculado)	Em aberto, Recebida ou Vencida

Indicadores do módulo

- Total a receber no período
- Contas recebidas
- Contas em aberto
- Contas vencidas

5. Centro de Despesas

Organização e categorização das despesas da empresa, independente de contas a pagar vinculadas a fornecedores.

Funcionalidades

- Cadastro de categorias de despesa
- Cadastro e lançamento de despesas
- Histórico de despesas por categoria e por período

Ficha de informações — Categoria de Despesa

Campo	Tipo	Obrigatório	Observações
Nome	Texto	Sim	Ex.: Aluguel, Marketing, Folha de pagamento
Tipo	Seleção	Sim	Fixa ou Variável
Descrição	Texto	Não	Detalhamento opcional

Ficha de informações — Despesa

Campo	Tipo	Obrigatório	Observações
Categoria	Relacionamento (Categoria de Despesa)	Sim	-
Descrição	Texto	Sim	-
Valor	Decimal (moeda)	Sim	-
Data	Data	Sim	Data de referência da despesa
Forma de pagamento	Relacionamento	Não	-
Conta bancária	Relacionamento	Não	Gera movimentação financeira quando informada
Recorrente	Booleano	Não	Indica se a despesa se repete mensalmente

Indicadores do módulo

- Gastos por categoria (período)
- Gastos mensais (total)
- Comparativo mensal (mês atual x mês anterior)

6. Produtos

Cadastro e controle de estoque dos produtos comercializados pela empresa.

Funcionalidades

- Cadastro de produtos e categorias
- Controle de estoque e estoque mínimo
- Definição de preço de compra e venda

Ficha de informações — Categoria de Produto

Campo	Tipo	Obrigatório	Observações
Nome	Texto	Sim	Ex.: Eletrônicos, Alimentos
Descrição	Texto	Não	-

Ficha de informações — Produto

Campo	Tipo	Obrigatório	Observações
Nome	Texto	Sim	-
Código/SKU	Texto	Sim	Identificador único do produto
Categoria	Relacionamento (Categoria de Produto)	Sim	-
Descrição	Texto	Não	-
Unidade de medida	Seleção	Sim	Ex.: Un, Kg, Cx, L
Preço de compra	Decimal (moeda)	Sim	Custo de aquisição
Preço de venda	Decimal (moeda)	Sim	-
Margem de lucro	Decimal (%)	Sim (calculado)	Calculada a partir de compra e venda
Estoque atual	Número	Sim (calculado)	Atualizado por movimentações de pedidos e entradas
Estoque mínimo	Número	Sim	Usado para alerta de estoque baixo
Fornecedor principal	Relacionamento (Fornecedor)	Não	-
Imagem	Arquivo	Não	-
Status	Booleano	Sim	Ativo ou inativo

7. Pedidos

Registro de vendas realizadas para clientes, com controle de itens, pagamento e status.

Funcionalidades

- Criação de novo pedido com múltiplos itens

- Aplicação de descontos
- Definição de forma de pagamento
- Histórico de pedidos e cancelamento

Ficha de informações — Pedido

Campo	Tipo	Obrigatório	Observações
Cliente	Relacionamento (Cliente)	Sim	-
Data do pedido	Data	Sim	-
Vendedor/usuário responsável	Relacionamento (Usuário)	Sim	-
Subtotal	Decimal (moeda)	Sim (calculado)	Soma dos itens antes do desconto
Desconto geral	Decimal (moeda ou %)	Não	Aplicado sobre o subtotal
Valor total	Decimal (moeda)	Sim (calculado)	Subtotal menos desconto
Forma de pagamento	Relacionamento	Sim	-
Status	Seleção	Sim	Aberto, Pago, Cancelado ou Entregue
Observações	Texto	Não	-

Ficha de informações — Item do Pedido

Campo	Tipo	Obrigatório	Observações
Pedido (vínculo)	Relacionamento	Sim	-
Produto	Relacionamento (Produto)	Sim	-
Quantidade	Número	Sim	Reduz o estoque do produto ao confirmar o pedido
Preço unitário	Decimal (moeda)	Sim	Preenchido a partir do cadastro do produto, editável
Desconto do item	Decimal (moeda ou %)	Não	-
Subtotal do item	Decimal (moeda)	Sim (calculado)	Quantidade x preço unitário, menos desconto

Indicadores do módulo

- Pedidos do dia (quantidade e valor)
- Valor total vendido no período
- Ticket médio

8. Clientes

Cadastro de clientes da empresa, com dados de contato e histórico de relacionamento comercial.

Funcionalidades

- Cadastro de dados pessoais/empresariais e endereço
- Cadastro de contatos
- Consulta de histórico de compras

Ficha de informações — Cliente

Campo	Tipo	Obrigatório	Observações
Tipo	Seleção	Sim	Pessoa Física ou Pessoa Jurídica
Nome/Razão social	Texto	Sim	-
CPF/CNPJ	Texto	Sim	Validação de formato conforme o tipo
Telefone	Texto	Não	-
E-mail	Texto	Não	-
Data de nascimento/abertura	Data	Não	-
Status	Booleano	Sim	Ativo ou inativo
Observações	Texto	Não	-

Ficha de informações — Endereço do Cliente

Campo	Tipo	Obrigatório	Observações
Cliente (vínculo)	Relacionamento	Sim	-
CEP	Texto	Não	-
Logradouro, número, complemento	Texto	Não	-
Bairro, cidade, UF	Texto	Não	-
Tipo de endereço	Seleção	Não	Cobrança ou Entrega

Indicadores do módulo

- Ranking de clientes (por valor comprado)
- Clientes inativos
- Histórico de compras por cliente

9. Fornecedores

Cadastro de fornecedores da empresa, com dados de contato, produtos fornecidos e histórico de compras.

Funcionalidades

- Cadastro de dados e contatos
- Vínculo com produtos fornecidos

- Consulta de histórico de compras

Ficha de informações — Fornecedor

Campo	Tipo	Obrigatório	Observações
Razão social	Texto	Sim	-
CNPJ/CPF	Texto	Sim	-
Contato principal	Texto	Não	Nome da pessoa de contato
Telefone	Texto	Não	-
E-mail	Texto	Não	-
Endereço	Texto	Não	-
Condições de pagamento	Texto	Não	Ex.: 30/60/90 dias
Status	Booleano	Sim	Ativo ou inativo

Indicadores do módulo

- Compras por fornecedor (período)
- Histórico de preços por produto/fornecedor

10. Indicadores

Painéis analíticos que consolidam dados dos demais módulos para apoiar decisões gerenciais.

Funcionalidades

- Financeiro: Fluxo de Caixa, Entradas x Saídas, Contas a Pagar, Contas a Receber, Despesas por Categoria
- Vendas: Vendas por período, por cliente, produtos mais/menos vendidos, ticket médio, formas de pagamento
- Estoque: Estoque atual, estoque baixo, produtos sem movimentação, valor do estoque
- Clientes: Ranking, clientes inativos, histórico de compras
- Fornecedores: Compras por fornecedor, histórico de preços
- Lucro: Lucro por produto, por período e margem de lucro

Ficha de informações — Filtro de Indicador (parâmetro de consulta, não é uma entidade persistida)

Módulo somente leitura: consome dados de Financeiro, Pedidos, Produtos, Clientes e Fornecedores conforme os filtros abaixo.

Campo	Tipo	Obrigatório	Observações
Período	Data (intervalo)	Sim	Padrão: mês corrente
Categoria/Cliente/ Fornecedor/Produto	Relacionamento	Não	Filtros específicos por tipo de indicador
Tipo de visualização	Seleção	Não	Gráfico de barras, linha, pizza ou tabela

11. Insights

Geração automática de alertas com base em regras de negócio, para antecipar problemas e oportunidades.

Funcionalidades

- Alertas de contas vencidas
- Alertas de estoque baixo
- Alertas de produto sem venda
- Alertas de cliente inativo
- Alertas de aumento de despesas
- Alertas de queda no faturamento
- Alertas de crescimento nas vendas

Ficha de informações — Alerta (Insight)

Campo	Tipo	Obrigatório	Observações
Tipo de alerta	Seleção	Sim	Conforme lista de funcionalidades acima
Condição de disparo	Texto/Regra	Sim (configurável)	Ex.: conta vencida há mais de 5 dias
Entidade relacionada	Relacionamento	Não	Conta, Produto, Cliente etc., conforme o tipo
Mensagem	Texto	Sim (gerado)	Texto exibido ao usuário
Data de geração	Data/Hora	Sim	-
Status	Seleção	Sim	Lido ou Não lido
Destinatário	Relacionamento (Usuário)	Não	Se vazio, visível a todos os usuários da empresa

12. Cadastros

Cadastros de apoio, utilizados como base para os demais módulos do sistema.

Funcionalidades

- Usuários
- Empresas
- Formas de pagamento
- Categorias de produtos
- Categorias de despesas
- Bancos e contas bancárias

Ficha de informações — Usuário

Campo	Tipo	Obrigatório	Observações
Nome	Texto	Sim	-
E-mail	Texto	Sim	Usado para login
Senha	Texto (hash)	Sim	Armazenada com criptografia

Campo	Tipo	Obrigatório	Observações
Papel (role)	Seleção	Sim	Ex.: Administrador, Financeiro, Vendedor
Empresa vinculada	Relacionamento (Empresa)	Sim	Cenário multiempresa
Status	Booleano	Sim	Ativo ou inativo

Ficha de informações — Empresa

Campo	Tipo	Obrigatório	Observações
Razão social	Texto	Sim	-
Nome fantasia	Texto	Não	-
CNPJ	Texto	Sim	-
Endereço	Texto	Não	-
Logo	Arquivo	Não	-
Configurações fiscais	Texto/JSON	Não	Regime tributário, dados para emissão de NF-e

Ficha de informações — Forma de Pagamento

Campo	Tipo	Obrigatório	Observações
Nome	Texto	Sim	Ex.: Dinheiro, Pix, Cartão de crédito, Boleto
Tipo	Seleção	Sim	À vista ou Parcelado
Taxa (%)	Decimal (%)	Não	Aplicável a cartões

Ficha de informações — Banco

Campo	Tipo	Obrigatório	Observações
Nome	Texto	Sim	-
Código do banco	Texto	Não	Código FEBRABAN, se aplicável

13. Escopo do MVP

O escopo consolidado do MVP contempla os módulos: Dashboard, Financeiro, Contas a Pagar, Contas a Receber, Centro de Despesas, Produtos, Pedidos, Clientes, Fornecedores, Indicadores, Insights e Cadastros.

Recomendação de sequenciamento de desenvolvimento:

- 1ª fase — Base: Cadastros (Usuários, Empresas, Formas de Pagamento, Bancos)
- 2ª fase — Operacional: Produtos, Clientes, Fornecedores
- 3ª fase — Financeiro: Contas Bancárias, Contas a Pagar, Contas a Receber, Centro de Despesas
- 4ª fase — Vendas: Pedidos
- 5ª fase — Consolidação: Dashboard, Indicadores e Insights